

LongConspectWriter: автоматическая генерация структурированных конспектов лекций на потребительском GPU

Александр Волжанин

[GitHub](#) · [Статья на Habr](#)

июнь 2026

Аннотация

Автоматическая генерация структурированных академических конспектов из аудиозаписей лекций по точным и естественным наукам затруднена для локальных малых языковых моделей (small language models, SLM). Транскрипт лекции продолжительностью $\approx 1,5$ ч составляет около 15–20 тыс. токенов и формально уместается в контекстное окно современных локальных SLM, однако при обработке такого контекста single-call SLM систематически деградируют: теряют фрагменты из середины последовательности, не удерживают структуру и галлюцинируют термины и формулы. Это проявление эффекта [Lost in the Middle](#): точность извлечения информации описывает U-образную кривую — высока на краях контекста и падает в середине; в наших условиях используются SLM, поведение которых так же описано в статье, и оно характеризуется выраженным забыванием не только из середины, но также и из начала контекста. Более того, на бюджете 8 ГБ VRAM single-call длинного транскрипта практически неприменим, что делает декомпозицию не оптимизацией, а необходимым условием работоспособности.

В работе предлагается проект LongConspectWriter (далее LCW) — мультиагентный пайплайн, в котором ни один вызов LLM не получает на вход полный транскрипт, а структура конспекта формируется до начала синтеза конспекта. Ключевой механизм — двухуровневое планирование поверх двухуровневой семантической кластеризации: транскрипт сегментируется на хронологически непрерывные локальные кластеры, для каждого генерируется краткое описание, по множеству описаний строится глобальный план глав, после чего локальные кластеры распределяются по главам. Синтез выполняется почанково по схеме MapReduce с реестром именованных сущностей, предотвращающим повторный ввод понятий.

Для оценки качества сформулирован набор из семи парадигм конспектирования, использованный как система критериев. В предварительной оценке по методологии LLM-as-a-judge LCW набирает в среднем 6,87/10 против 8,84/10 у Gemini 3.1 Pro (на момент проведения работы являющейся SOTA-моделью от Google), обрабатывавшей тот же транскрипт, но за один вызов, что соответствует $\approx 78\%$ качества эталона при нулевой стоимости инференса и полностью локально. При этом наименее пострадали именно те характеристики, которые наиболее уязвимы к деградации на длинном контексте, — информационная полнота и самодостаточность показали наибольшую близость к эталону (с оговоркой, что высокая полнота отчасти заложена самим методом, — см. раздел 7), тогда как основное отставание пришлось на фактическую точность и математическую строгость, ограниченные скорее размером локальной модели, а не архитектурой пайплайна. Визуализация низка у обеих систем — автоматический выбор уместного типа графика и его корректная интеграция в конспект, объективно сложная задача вне зависимости от размера модели.

Выборка на тестах составила всего 10 лекций, поэтому делать статистические выводы нельзя: число 78% следует трактовать как предварительный результат для построения гипотезы. Исходный код, датасет тестирования и тесты доступны под лицензией MIT на [GitHub](#).

Содержание

1. Введение	2
2. Обзор существующих решений	4
3. Метод	5
3.1. Обзор	5
3.2. Транскрибация	5
3.3. Локальная семантическая кластеризация	5
3.4. Двухуровневое иерархическое планирование	7
3.5. Глобальная кластеризация	8
3.6. Синтез конспекта	8
3.7. Генерация визуализаций	9
4. Аппаратно-ориентированный дизайн	10
5. Парадигмы конспектирования	10
6. Экспериментальная методология	11
7. Результаты	11
8. Ограничения	13
9. Заключение	13

1. Введение

Природа деградации на длинном контексте изучена [Liu и соавт.](#): на задачах multi-document QA и key-value retrieval точность извлечения информации описывает U-образную кривую — она высока на краях контекста и падает в середине окна. В оригинальной работе это показано на моделях MPT-30B-Instruct, LongChat-13B, GPT-3.5-Turbo и Claude-1.3. В нашей постановке используется малая (8B) квантизованная модель. К самой деградации на бюджете 8 ГБ VRAM добавляется ресурсное ограничение: single-call длинного транскрипта на целевом железе вдобавок неосуществим по памяти и скорости (детальный разбор — ниже). Это соображение об осуществимости вторично по отношению к самой деградации и лишь усиливает вывод: декомпозиция здесь не оптимизация, а необходимое условие.

Осуществимость single-call на целевом железе. Прямая обработка полного транскрипта (~15–20 тыс. токенов) целевой моделью T-lite (8B) на стенде RTX 3050 8 ГБ нереализуема по двум различным причинам, в зависимости от формата исполнения. В неквантованном формате (transformers, fp16) одни только веса 8B-модели превышают 8 ГБ VRAM, и запуск падает с ошибкой нехватки памяти (OOM) ещё до начала генерации. В квантованном формате (GGUF Q5_K_M через llama-cpp-python 0.3.20) модель размещается в памяти, однако рабочая конфигурация пайплайна использует контекстное окно 8192 токена, в которое полный транскрипт (15–20 тыс. токенов) не помещается; single-call потребовал бы кратно большего `n_ctx`, чей KV-кеш поверх весов на 8 ГБ либо не умещается, либо вынуждает вынести часть слоёв в RAM — отчего скорость генерации падает до единиц токенов в секунду. То есть в GGUF single-call не столько невозможен, сколько практически неприемлем по времени. Переход к существенно меньшей модели ради сокращения требований к памяти даёт непригодный выход. Практическое следствие: на данном бюджете single-call не является работоспособной альтернативой, поэтому корректным quality-эталонном служит облачная модель (раздел 6), а не single-call той же модели.

Отсюда центральная идея работы — строить структуру конспекта до синтеза текста. Интуиция проста: если модель не способна надёжно удержать весь транскрипт целиком, ей и не следует видеть его целиком — сначала по коротким фрагментам составляется «карта» лекции, и лишь затем по этой карте, фрагмент за фрагментом, пишется текст.

Простейшая форма такой декомпозиции — наивный map-reduce, независимое сжатие чанков с последующей конкатенацией — на целевом железе осуществима, но для лекции неудовлетворительна: каждый вызов видит лишь свой короткий фрагмент и не имеет представления о лекции в целом, поэтому итог распадается на несвязные куски с повторяющимися определениями и без контроля полноты охвата. Узкое место здесь — не длина контекста отдельного вызова (каждый чанк и так короткий), а несогласованность вызовов между собой: ни один из них не знает ни своего места в лекции, ни того, что уже сказано в остальных. LCW устраняет именно это: сначала из коротких описаний кластеров собирается план глав — карта всей лекции, — и только затем под этот план, глава за главой, синтезируется текст. План и есть то общее знание, которого лишён наивный map-reduce; при этом полный транскрипт по-прежнему не попадает ни в один вызов.

Настоящая работа рассматривает задачу в постановке, фиксирующей три жёстких ограничения одновременно: полностью локальное исполнение без облачных API или платных подписок; 8 ГБ VRAM — самый распространённый объём видеопамати среди пользователей Steam (~27%, [Steam Hardware Survey](#), апрель 2026); устойчивость к длинному контексту для лекций продолжительностью 1,5 ч.

Вклад работы состоит в следующем:

- (1) предложен LongConspectWriter — полностью локальный мультиагентный пайплайн, в котором ни один вызов LLM не получает на вход полный транскрипт, а структура конспекта формируется до начала синтеза текста;
- (2) предложен механизм двухуровневого иерархического планирования поверх хронологически ограниченной семантической кластеризации, позволяющий строить глобальный план конспекта без подачи полного транскрипта на вход;
- (3) сформулирован набор из семи парадигм конспектирования как система требований и критериев оценки качества академических конспектов;
- (4) проведена экспериментальная оценка по методологии LLM-as-a-judge на 10 лекциях

из пяти предметных доменов с сопоставлением с облачным SOTA-эталонном.

2. Обзор существующих решений

Существующие подходы к автоматическому конспектированию аудиолекций распадаются на две группы, каждая из которых оставляет нерешённой хотя бы одно из трёх ограничений, зафиксированных во введении.

Локальные пайплайны без планирования структуры. Существуют [полностью локальные решения транскрибации и суммаризации аудио](#), не зависящие от облачных API и решающие ограничение локальности. Однако такие пайплайны строят конспект прямой суммаризацией транскрипта без предварительного построения структуры, и на длинном транскрипте наследуют ту же деградацию на длинном контексте, что и single-call подача: устойчивость к длинному контексту остаётся нерешённой.

Облачные сервисы генерации учебных материалов. [Облачные решения](#) опираются на крупные модели с большим контекстным окном и достигают высокого качества, но нарушают требования локальности и нулевой стоимости инференса: обработка требует передачи записи в сторонний сервис и платной подписки.

Положение настоящей работы. LCW занимает промежуточную позицию: сохраняя полную локальность первой группы, он отказывается от прямой суммаризации длинного транскрипта и строит структуру конспекта до синтеза текста, адресуя тем самым именно ту деградацию на длинном контексте, которую локальные решения первой группы оставляют без внимания. Иными словами, структура здесь — не оформление, а защитный механизм: она фиксирует, что и где должно быть сказано, ещё до того, как SLM начнёт писать текст и получит возможность что-то потерять. Новизна работы — не в почанковой обработке как таковой (это базовое свойство иерархических суммаризаторов) и не в кластеризации по отдельности, а в конкретной связке: глобальный план конспекта строится из кратких описаний кластеров до синтеза текста; локальная кластеризация хронологически ограничена матрицей связности; межчанковая связность поддерживается реестром именованных сущностей без подачи полного контекста; и весь конвейер спроектирован под ограничение 8 ГБ VRAM.

Предложенный подход примыкает к линии работ по декомпозиции длинного контекста, в которой можно выделить три направления. Иерархическая и рекурсивная суммаризация сжимает текст снизу вверх: [Wu и соавт.](#) рекурсивно суммаризируют книги, сначала сжимая небольшие фрагменты, а затем — их сводки, а [RAPTOR](#) рекурсивно кластеризует фрагменты и строит дерево сверток. Мультиагентные схемы распределяют длинный вход между вызовами: в [Chain of Agents](#) воркеры читают чанки последовательно, передавая друг другу сжатое сообщение, а финальный агент собирает ответ. Линия plan-before-write строит структуру до текста: [STORM](#) сначала формирует план статьи, а затем пишет под него из найденных источников. Этап локальной кластеризации при этом родственен классической сегментации текста на подтемы ([TextTiling](#), [Hearst](#)), причём двухуровневая структура «подтемы внутри тем» прямо перекликается с нашим разделением на локальные и глобальные кластеры.

На этом фоне новизна LCW носит не алгоритмический, а системно-методический характер, и её уместно разложить на три уровня. Во-первых, инженерный: ни одна из перечисленных работ не нацелена на жёсткий бюджет 8 ГБ VRAM при полностью локальной SLM ([RAPTOR](#) и [STORM](#) опираются на крупные облачные модели), и вкладом служит сама осуществимость такой связки. Во-вторых, методический: в отличие от семантической кла-

стеризации RAPTOR, объединяющей близкие фрагменты независимо от их позиции, LCW использует временную монотонность лекции как структурный приор — и кластеризация (матрица связности), и приписывание кластеров к главам (хронологическое сглаживание переходов) ограничены ходом времени, благодаря чему глобальный план восстанавливается из кратких описаний кластеров без подачи полного транскрипта. В-третьих, эмпирический: предложен набор из семи парадигм как система критериев и сформулирована проверяемая гипотеза о разделении осей качества (раздел 9). Алгоритмически новых компонентов работа не предлагает: вклад — в их увязке под три одновременных ограничения, зафиксированных во введении.

3. Метод

3.1. Обзор

Пайплайн организован в три последовательных блока: транскрибация и двухуровневая кластеризация с планированием; синтез текста; генерация визуализаций и сборка PDF. Каждый блок сохраняет промежуточный результат на диск и передаёт следующему блоку путь к артефакту, а не данные в памяти; инженерное следствие этого решения для управления VRAM рассматривается в разделе 4.

Транскрипт лекции продолжительностью $\approx 1,5$ ч имеет объём порядка 15–20 тыс. токенов. Все текстовые агенты пайплайна (планировщики, синтезатор, экстрактор, планировщик визуализаций) используют единую модель [T-lite-it-2.1](#) в квантизации GGUF Q5_K_M (файл `T-lite-it-2.1-Q5_K_M.gguf`) с контекстным окном 8192 токена (у планировщика визуализаций — 4096) через [llama-cpp-python](#) (версия 0.3.20); агент генерации кода визуализаций — [Qwen2.5-Coder-7B-Instruct](#) в квантизации Q6_K (контекст 4096 токенов). Генерация у текстовых агентов жадная (температура 0, `min_p` 0,15, `repeat_penalty` 1,05; у планировщика визуализаций — 1,1). Общая схема пайплайна приведена на рис. 1.

Как показано [во введении](#), оба наивных подхода на целевом железе неосуществимы: `single-call` неосуществим по памяти и скорости (раздел 1), а голый `map-reduce` запускается, но даёт фрагментарный конспект без глобальной структуры. Поэтому LCW надстраивает над почанковой обработкой два механизма: двухуровневое планирование задаёт глобальную структуру до синтеза, а реестр сущностей убирает дублирование. Тем самым декомпозиция решает проблему осуществимости, а планирование и реестр — проблему качества конспекта.

3.2. Транскрибация

Аудиозапись преобразуется в текст моделью `Whisper large-v3-turbo` в реализации [faster-whisper](#) (формат `CTranslate2`). Перед запуском подаётся системный промпт, варьируемый в зависимости от предметного домена лекции, что повышает корректность распознавания доменной терминологии. Результатом этапа является сырой транскрипт.

3.3. Локальная семантическая кластеризация

Транскрипт сегментируется на локальные кластеры — хронологически непрерывные смысловые блоки по одной микротеме. Предложения транскрипта (разбиение — библиотека `razdel`) векторизуются моделью [rubert-tiny2](#) на CPU, а кластеризация выполняется [агломеративным алгоритмом](#) с косинусной метрикой, полной связью (`complete linkage`) и

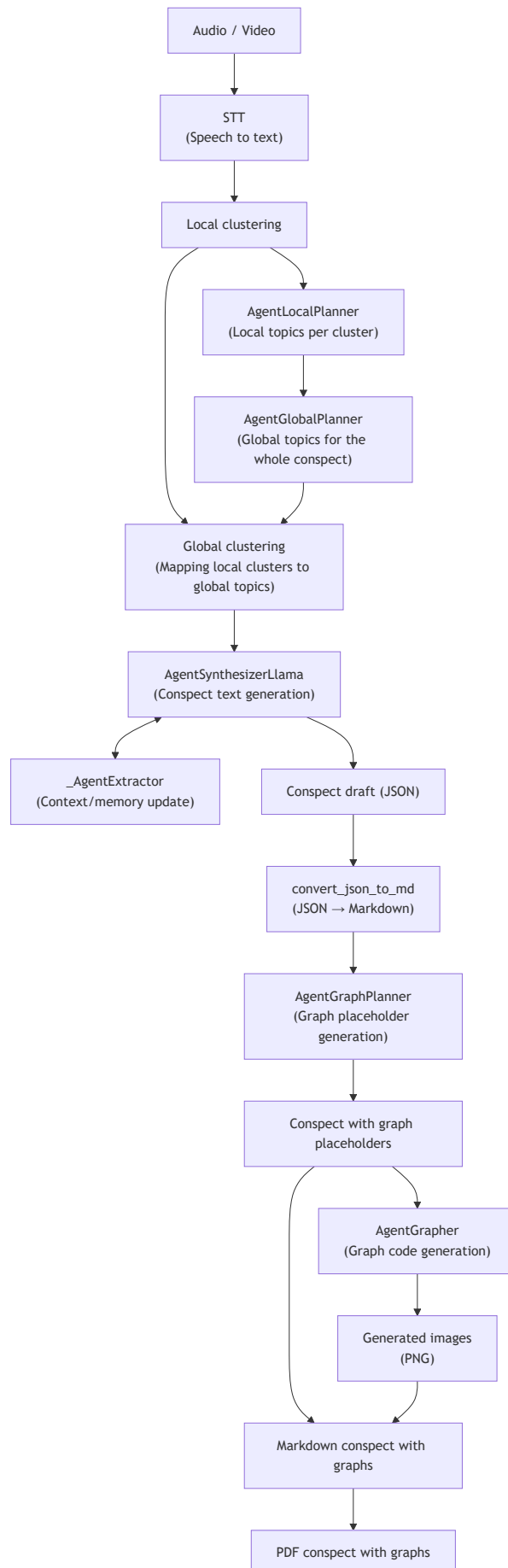


Рис. 1. Общая архитектура мультиагентного пайплайна LCW.

порогом расстояния 0,5. Число кластеров заранее не задаётся, а определяется этим порогом.

Принципиальным является ограничение хронологии. Несмежные по времени фрагменты могут оказаться семантически близкими — например, при повторном обращении лектора к определению, — и стандартная кластеризация объединила бы их, нарушив временной порядок. Чтобы этого избежать, агломеративный алгоритм запускается с матрицей связности, которая разрешает объединять только соседние по времени фрагменты и запрещает слияние хронологически отдалённых. Матрица разрешает связь только с непосредственными соседями по времени (две побочные диагонали — матрица смежности).

```
# Матрица связности: каждое предложение соединено только
# с непосредственными соседями по времени (i-1 и i+1)
if self._gen_config.turn_on_connectivity:
    connectivity = np.eye(n_samples, k=1) + np.eye(n_samples, k=-1)

local_clusterer = AgglomerativeClustering(
    n_clusters=None,
    distance_threshold=self._gen_config.threshold, # 0.5
    metric=self._gen_config.metric, # cosine
    linkage=self._gen_config.linkage, # complete
    connectivity=connectivity,
) # упрощено
```

В результате каждый локальный кластер представляет собой непрерывную последовательность идущих подряд предложений.

Условный пример для иллюстрации. Пусть в начале лекции подряд идут предложения «Рассмотрим определение предела последовательности», «Последовательность сходится, если...» и «Иначе говоря, начиная с некоторого номера все члены...». Они семантически близки и хронологически смежны, поэтому попадают в один локальный кластер. Если же спустя сорок минут лектор возвращается к теме («вспомним определение предела, с которого мы начинали»), этот фрагмент семантически близок первому кластеру, но хронологически от него удалён — матрица связности запрещает их слияние, и повтор остаётся в своём, более позднем кластере, сохраняя ход лекции.

3.4. Двухуровневое иерархическое планирование

Локальное планирование. Каждый локальный кластер независимо передаётся агенту LocalPlanner, который формирует одну краткую строку с описанием того, о чём этот кластер. Агент обрабатывает ровно один кластер за вызов и решает ровно одну задачу. Это согласуется с гипотезой из введения: SLM надёжно решает локальную задачу на коротком входе, тогда как глобальная задача на длинном входе ей недоступна без деградации.

Условный пример: для кластера из предыдущего пункта LocalPlanner мог бы вернуть строку вида «Определение предела последовательности и его неформальная интерпретация».

Глобальное планирование. Полученные описания собираются в единый список, который — и только он, без исходного текста лекции — передаётся агенту GlobalPlanner. Агент строит из этих описаний единый иерархический план глав всего конспекта с точными заголовками разделов в формате JSON. Поскольку на вход подаются сводные описания, а

не транскрипт, объём входа мал и уместается в контекстное окно даже для многочасовых лекций.

Продолжая тот же условный пример: входом GlobalPlanner служит список из нескольких десятков строк вида «Определение предела последовательности и его неформальная интерпретация», «Арифметические свойства пределов», «Примеры вычисления пределов», по которому агент мог бы собрать главу «Предел последовательности» с соответствующими подразделами. Полный текст лекции при этом ни на каком шаге планирования модели не предъявляется.

3.5. Глобальная кластеризация

Каждый локальный кластер приписывается к одной из глав плана. Для сопоставления многоязычной моделью [multilingual-e5-small](#) векторизуются, с одной стороны, главы плана (заголовок вместе с описанием, с префиксом `query:`), с другой — сам текст каждого локального кластера (с префиксом `passage:`); кластер относится к той главе, эмбединг которой ближе всего к нему по косинусной мере.

Поверх такого попарного выбора применяется хронологическое сглаживание: поскольку главы плана упорядочены по ходу лекции, переходы между соседними кластерами должны быть монотонными. Предложенный моделью «прыжок назад» к более ранней главе игнорируется, а одиночный «прыжок вперёд» принимается как состоявшийся переход только при подтверждении соседними кластерами — иначе он трактуется как выброс эмбедингера и отбрасывается. Объединение всех локальных кластеров, приписанных к одной главе, образует глобальный кластер — тематически сгруппированный набор фрагментов транскрипта, готовый к синтезу.

3.6. Синтез конспекта

На данном этапе применяется схема MapReduce: глобальный кластер делится на чанки фиксированного размера с перекрытием (≈ 1433 токена при перекрытии ≈ 512 токенов — 0,7 и 0,25 от лимита генерации в 2048 токенов; длина измеряется токенизатором самой модели), и текст пишется почанково агентом Synthesizer.

В каждом отдельном вызове синтезатор видит только четыре вещи: заголовок текущей главы; текущий чанк транскрипта; «хвост» предыдущего чанка — последние 50 слов для сшивания границы; и имена уже введённых в конспект сущностей.

```
seen_str = ", ".join(already_seen_themes) or "Ничего ещё не было разобрано."
combined_context = (
    f"[УЖЕ РАЗОБРАНО]: {seen_str}\n"           # реестр имён сущностей
    f"[КОНЕЦ ПРЕДЫДУЩЕГО ЧАНКА]: {last_tail}" # хвост предыдущего чанка
)
prompt = self._build_prompt(
    chunk=chunk,                               # текущий чанк транскрипта
    cluster_topik=topik,                       # заголовок текущей главы
    previous_context=combined_context,
) # упрощено
```

Реестр имён поддерживается вспомогательным агентом Extractor: после каждого написан-

ного фрагмента он извлекает из него имена новых сущностей и дописывает их в глобальный реестр, общий для всей лекции. Синтезатор видит этот реестр и не вводит повторно уже объяснённые понятия, что устраняет дублирование определений.

```
# после каждого написанного чанка
last_tail = " ".join(synthesize_chunk.split()[-50:]) # хвост для сшивания
extracted = self._extractor.run(synthesizer_chunk=synthesize_chunk)
for term in extracted.get("extracted_entities", []):
    already_seen_themes.add(term.lower()) # дописываем в реестр
```

Зная своё положение в документе через глобальный план и состояние реестра сущностей, агент порождает сухой академический текст без мета-нарратива и артефактов устной речи. Итог блока — конспект в формате JSON (ключ — тема главы, значение — её текст), экспортируемый в Markdown.

Здесь уместно ответить на естественное возражение: если синтезатор и так сжимает предыдущий контекст до хвоста и списка имён, почему не применить тот же приём напрямую ко всей лекции, минуя планирование и кластеризацию? Потому что структуру несёт не этот перенос, а два входа, которые синтезатор получает выше по пайплайну. Во-первых, каждый вызов пишется под заголовок конкретной главы из глобального плана — без планирования у синтезатора нет ни заголовка, ни скелета документа, под которые он размещает текст. Во-вторых, на вход подаётся не сырая хронология, а транскрипт, уже разрезанный по главам: хвост для сшивания границы сбрасывается на старте каждой главы, поэтому он никогда не склеивает два разных топики, тогда как при линейной нарезке границы чанков попадали бы в середину темы. Сам реестр имён при этом — намеренно минимальный инструмент против повторного ввода понятий: он глобален на всю лекцию и несёт только имена сущностей без определений (см. раздел 8), то есть подавляет дублирование, но не задаёт структуру. Иными словами, столь скудного переноса достаточно именно потому, что глобальную структуру задают план и выровненная по главам сегментация, а не он сам.

3.7. Генерация визуализаций

Визуализация вынесена в отдельный финальный блок и разделена на две роли.

Агент GraphPlanner анализирует готовый Markdown и расставляет плейсхолдеры вида [GRAPH_TYPE: ...] в семантически уместных местах; точки вставки определяются нормализованным поиском по цитатам, текст конспекта при этом не изменяется.

Агент Grapher для каждого плейсхолдера порождает исполняемый Python-скрипт построения изображения. Скрипт выполняется в изолированном подпроцессе с урезанным окружением и таймаутом 15 секунд. Это единственный компонент системы, использующий самокоррекцию: при ошибке исполнения агент получает stderr и предыдущий неработающий код и повторяет генерацию, повышая температуру на 0,1 при каждой попытке (старт с 0), — всего до трёх попыток.

```
original_temp = self._gen_config.temperature # 0
for attempt in range(self._app_config.re_try_count): # до 3 попыток
    self._gen_config.temperature = original_temp + attempt * 0.1 # 0 → 0.1 → 0.2
    code = self._generate_graph_code(description, target, error_message, bad_code)
```

```
is_success, stderr_text = self._code_call(code, target, script_path)
if is_success:
    break
bad_code, error_message = code, f"Твой код упал с ошибкой:\n{stderr_text}"
```

Успешно сгенерированные изображения сохраняются в PNG. На финальном этапе плейсхолдеры заменяются ссылками на изображения, и документ конвертируется в постраничный PDF.

4. Аппаратно-ориентированный дизайн

Суммарный объём весов всех моделей пайплайна (STT, две embedding-модели, две LLM) превышает 8 ГБ, поэтому одновременное резидентное размещение в видеопамяти невозможно. Применяется строгая политика: в каждый момент времени в VRAM находится не более одного компонента. STT-компонент завершает транскрибацию и полностью выгружается до инициализации LLM-агентов, embedding-модель выгружается после кластеризации и т. д.

Технически эта политика обеспечивается изоляцией этапов по процессам: метод `run()` каждого этапа выполняется в отдельном дочернем процессе, который завершается по окончании этапа, — операционная система при этом освобождает всю занятую им память, включая VRAM. Именно поэтому блоки обмениваются данными через артефакты на диске (раздел 3.1): результат каждого этапа персистентен, и следующему процессу не требуется (и невозможно) удерживать модель предыдущего в памяти. Пиковое потребление VRAM определяется не суммой моделей, а наиболее ресурсоёмким компонентом. Ценой служит дополнительное время на загрузку/выгрузку.

Целевая конфигурация — GPU от 8 ГБ VRAM. На тестовом стенде NVIDIA RTX 3050 (8 ГБ) полный путь от аудиофайла до готового PDF для лекции продолжительностью ≈ 1 ч 30 мин завершается приблизительно за 35–40 минут.

5. Парадигмы конспектирования

Для формализации понятия «качественный конспект» сформулированы семь парадигм. Первые четыре задают структурно-содержательные требования, последние три — критерии практического качества. Следует оговорить ограничение: эти же парадигмы выступают и проектными ориентирами пайплайна, и критериями его оценки, поэтому высокие баллы по ним частично отражают согласованность цели и меры, а не только независимое подтверждение качества; сопоставление с эталоном, оцениваемым по тем же критериям, частично смягчает этот эффект.

- **P1. Модульная семантическая архитектура.** Конспект — не линейный пересказ, а структурированная база знаний с явными функциональными блоками (определения, теоремы, доказательства, следствия, примеры), допускающая навигацию без полного прочтения.
- **P2. Объективизация и деперсонализация.** Отсутствие мета-нарратива (приветствий, отступлений, оборотов «далее мы рассмотрим», «лектор переходит к») и артефактов устной речи; изложение оперирует фактами и логикой.
- **P3. Математическая и терминологическая строгость.** Формулы и символы — полноправный язык; недопустимо приблизительное словесное описание формул, требуется строгая нотация (стандарты LaTeX).

- **Р4. Пространственно-визуальная интеграция.** Где материал требует графического представления, должны присутствовать обозначенные места интеграции визуального контента с описанием изображаемого, а не вербальное описание сложных пространственных объектов.
- **Р5. Фактическая точность.** Воспроизведение фактов, определений и утверждений без домыслов; подстановка правдоподобных, но ошибочных формулировок недопустима — каждое утверждение должно содержаться в исходной лекции.
- **Р6. Информационная полнота.** Отражение смыслового содержания лекции, а не только корректность того, что попало в конспект; оценивается тяжесть пробелов.
- **Р7. Самодостаточность.** Готовый конспект понятен читателю, не присутствовавшему на лекции, и не требует обращения к исходной записи.

6. Экспериментальная методология

Метод оценки. Использована методология LLM-as-a-judge: модель-судья Qwen3 Max Preview оценивала конспект по специализированному промπτу, получая на вход исходный транскрипт и оцениваемый конспект (PDF).

Эталон. Gemini 3.1 Pro (на момент проведения работы это была SOTA-модель), обрабатывавшая тот же транскрипт single-call по детальному промπτу. Тем самым предложенная декомпозиционная локальная схема сопоставляется с прямой подачей полного контекста топовой облачной модели.

Датасет. 10 лекций, 5 предметных доменов по 2 лекции: алгоритмы, машинное обучение, математический анализ, биология, химия.

Промпт судьи. Транскрипт объявляется единственным источником истины: конспект оценивается строго относительно прозвучавшего, без привлечения внешних знаний и без штрафа за отсутствие того, чего лектор не говорил; пропуск организационных реплик, слов-паразитов и повторов ошибкой не считается. По каждой парадигме заданы определение, признаки нарушения и градуированная шкала 1–10. Для Р5 формируется структурированный перечень ошибок с делением на критические/значимые/незначительные; для Р6 судья предварительно составляет перечень ключевых смысловых единиц транскрипта и размечает каждую по статусу раскрытия (covered_deep / covered / partial / missed), и лишь затем выставляет балл. Ответ возвращается строго в JSON с обоснованием и цитатой по каждому пункту, что обеспечивает воспроизводимость разбора.

7. Результаты

Средний балл по семи парадигмам составил 6,87/10 для LCW против 8,84/10 для Gemini 3.1 Pro, что соответствует $\approx 78\%$ качества эталона при нулевой стоимости инференса и полном офлайн-режиме на GPU с 8 ГБ VRAM. Оценки получены за один прогон судьи на выборке из 10 лекций, поэтому приводимые значения не сопровождаются оценкой дисперсии и служат ориентиром, а не точным измерением (подробнее — раздел 8). Результаты по доменам приведены в табл. 1.

Таблица 1. Баллы по 10 лекциям с подытогами по доменам и общим средним (LCW / Gemini 3.1 Pro / % от эталона).

#	Тема	Домен	LCW	Gemini 3.1 Pro	LCW %
1	Введение. Базовые конструкции Python	Алгоритмы	8,14	8,57	95%

#	Тема	Домен	LCW	Gemini 3.1 Pro	LCW %
2	Алгебра логики. Ветвления	Алгоритмы	6,00	7,86	76%
	Алгоритмы		7,07	8,22	86%
3	Введение. Основные понятия и обозначения	Машинное обучение	5,43	8,43	64%
4	Метрические методы классификации	Машинное обучение	6,57	8,29	79%
	Машинное обучение		6,00	8,36	72%
5	Основные определения математического анализа	Мат. анализ	7,71	8,86	87%
6	Функция и её график. Множества целых и рациональных чисел	Мат. анализ	6,71	8,71	77%
	Мат. анализ		7,21	8,79	82%
7	Определение жизни. Микро- и макроэлементы	Биология	7,86	10,00	79%
8	Биологические мембраны. Вторичные метаболиты	Биология	7,00	9,57	73%
	Биология		7,43	9,79	76%
9	Основные понятия химии	Химия	8,00	9,71	82%
10	Атом водорода и многоэлектронные атомы	Химия	5,29	8,43	63%
	Химия		6,65	9,07	73%
	Среднее		6,87	8,84	78%

При двух лекциях на домен порядок доменов иллюстративен и не интерпретируется.

Детализация по семи парадигмам приведена в табл. 2 (формат ячеек — LCW / Gemini 3.1 Pro; номера лекций соответствуют табл. 1).

Таблица 2. Баллы по семи парадигмам P1–P7 для каждой из 10 лекций и среднее (LCW / Gemini 3.1 Pro).

Лекция	P1	P2	P3	P4	P5	P6	P7	Среднее
1	9/9	10/10	8/9	3/4	9/10	9/9	9/9	8,14/8,57
2	8/9	8/10	3/8	3/3	4/10	8/8	8/7	6,00/7,86
3	6/10	9/10	5/9	2/2	4/10	7/9	5/9	5,43/8,43
4	7/9	5/10	5/8	3/3	9/10	9/9	8/9	6,57/8,29
5	9/10	10/10	9/9	3/4	5/10	9/10	9/9	7,71/8,86
6	7/9	9/10	4/8	1/6	9/10	9/9	8/9	6,71/8,71
7	8/10	10/10	6/10	3/10	10/10	9/10	9/10	7,86/10,00
8	6/10	10/9	6/10	1/8	8/10	9/10	9/10	7,00/9,57
9	9/10	9/10	7/9	3/9	9/10	10/10	9/10	8,00/9,71
10	5/10	6/10	4/6	1/5	5/10	9/9	7/9	5,29/8,43
Среднее	7,4/9,6	8,6/9,9	5,7/8,6	2,3/5,4	7,2/10,0	8,8/9,3	8,1/9,1	6,87/8,84

Сильные стороны. Наибольшая близость к эталону — по информационной полноте (P6: 8,8 / 9,3) и самодостаточности (P7: 8,1 / 9,1). Это согласуется с центральной гипотезой: двухуровневое иерархическое планирование эффективно решает задачу полноты охвата («не упустить тему») даже при отсутствии сквозного контекста, и итоговый конспект читается без обращения к исходной записи. Существенно, что наиболее уязвимые к проблеме длинного контекста характеристики оказались наименее пострадавшими. Следует учитывать, что высокая полнота отчасти заложена самим методом — глобальный план по построению перечисляет темы, — поэтому близость к эталону по P6 и P7 лишь частично является независимым подтверждением качества, а не чисто неожиданным эффектом.

Слабые стороны. Наибольшее отставание от эталона — по фактической точности (P5: 7,2 / 10,0) и математической строгости (P3: 5,7 / 8,6). Обе характеристики чувствительны к размеру модели: квантизованная 8B-модель в формате GGUF уступает крупным облач-

ным моделям при работе со сложной математической нотацией и узкой терминологией. Это ограничение лежит на уровне модели и не устраняется архитектурой пайплайна.

Визуализация. Парадигма P4 показывает низкие значения у обеих систем (LCW: 2,3; Gemini: 5,4): автоматический выбор уместного типа графика и его корректная генерация представляют объективно сложную задачу вне зависимости от размера модели. Близкие к половине значения у SOTA-эталона указывают, что узким местом здесь является сама задача, а не только размер локальной модели.

8. Ограничения

Методологические. Применённая методология LLM-as-a-judge не калибровалась по человеческой экспертной оценке, поэтому модель-судья могла допускать ошибки. Судья использовался через веб-интерфейс (не через API) и запускался однократно на каждую лекцию, что не учитывает дисперсию оценок. Объём выборки (10 лекций) достаточен для первичной оценки, но недостаточен для статистически значимых выводов. Кроме того, не контролировались два известных смещения методологии LLM-as-a-judge: порядковый эффект (в каком порядке судье предъявлялись конспекты LCW и эталона) и влияние формата и длины (судья оценивал PDF, поэтому вёрстка и объём документа могли частично влиять на оценку содержания). По этим причинам количественные результаты раздела 7 следует трактовать как ориентировочные.

Дизайн сравнения. Эталон (Gemini 3.1 Pro, single-call) и LCW различаются по двум осям — размеру модели и архитектуре. Изолировать вклад именно декомпозиции сравнением «та же модель single-call против неё же в LCW» не удалось по объективной причине: single-call той же модели на целевом железе неосуществим (раздел 1). Поэтому количественный quality-эталон — облачная модель, а не single-call; полученные $\approx 78\%$ характеризуют близость к практически доступной альтернативе (облаку), но не доказывают причинного преимущества декомпозиции по качеству над single-call, который на 8 ГБ попросту недоступен. Сравнение LCW с другими осуществимыми на 8 ГБ декомпозициями (retrieval-подходы, скользящее окно) остаётся будущей работой.

Архитектурные. Реестр сущностей передаёт синтезатору только имена ранее введённых сущностей, но не их определения: агент «знает» о факте упоминания термина, но не располагает его исходной трактовкой, что в отдельных случаях способно порождать неточности изложения. Эффективное сжатие предыдущего контекста, превосходящее по информативности список имён, остаётся открытым вопросом. Кроме того, ошибки глобальной кластеризации могут привести к отнесению локального кластера не к той главе и попаданию фрагмента в семантически неподходящий раздел.

Инженерные и предметные. Кластеризация выполняется по сырой транскрибации без предварительной разметки, что может отражаться на качестве сегментации. Система оптимизирована исключительно для русскоязычных лекций по точным, естественным и техническим наукам; гуманитарные дисциплины и иные языки выходят за рамки решаемой задачи.

9. Заключение

Установлено. На потребительском GPU с 8 ГБ VRAM прямая (single-call) обработка длинной лекции практически неосуществима по ресурсному бюджету — памяти и скорости (раздел 1), поэтому декомпозиция, удерживающая контекст каждого вызова малым, выступает необходимым условием работоспособности, а не оптимизацией качества. Пред-

ложенный пайплайн LongConspectWriter реализует такую декомпозицию, запускается на этом железе и достигает $\approx 78\%$ качества облачной SOTA-модели Gemini 3.1 Pro при нулевой стоимости инференса и полной локальности обработки (оценка на 10 лекциях). Сверх простого map-reduce, дающего лишь осуществимость, за качество конспекта отвечают два механизма: двухуровневое планирование структуры до синтеза и реестр именованных сущностей.

Гипотеза. Предварительные данные позволяют выдвинуть — как проверяемое предположение, а не доказанное утверждение — гипотезу о разделении осей качества: архитектуры со структурой-до-синтеза восстанавливают преимущественно организационную составляющую (охват, структуру, самодостаточность) до уровня, близкого к крупной модели, тогда как разрыв концентрируется в содержательной точности (факты, формулы, терминология), которая ограничена ёмкостью модели и архитектурой не закрывается. Проще говоря, под «осями качества» понимаются два разных свойства конспекта: насколько он полон и хорошо организован — и насколько он точен в деталях; декомпозиция, по-видимому, помогает первому, но не второму. Объёма выборки (10 лекций) достаточно для демонстрации работоспособности, но недостаточно для подтверждения этой гипотезы как обобщения.